FML2ILV – Federated Machine Learning

FL Algorithms in Practice

Assoc. Prof. Alexander Jung

8 April 2026 – IMC Krems

Glossary **Book** **GitHub**

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

Your Feedback After Session 2

Thank you for the 18 responses!

Top themes:

- Math (fixed-point iterations,. . .) hard to follow
- Theory-to-code gap feels abrupt
- Pacing too fast at the end; need more step-by-step build-up
- Request for recaps and repetition of key concepts

Today's plan: hands-on exercise session that bridges the theory–code gap.
We work through concrete examples with actual numbers.

Today's Agenda

1. **Quick recap** of the FL algorithm workflow (15 min)
2. **Demo 1:** GD as a fixed-point iteration – single device, real data (30 min)
3. **Demo 2:** feature normalization – reducing κ and the contraction rate (15 min)
4. **Demo 3:** two-device FL – pseudo-label exchange (30 min)

All Python scripts are in the `demos/` folder on GitHub.

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

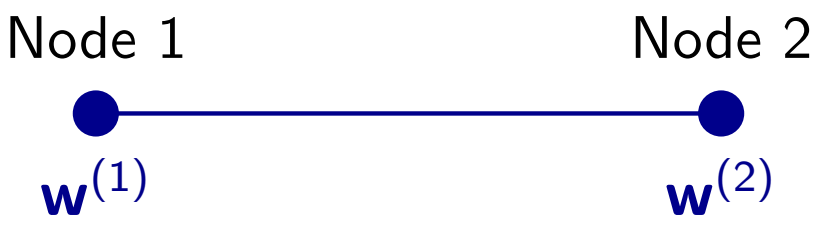
Our Setting: A Toy FL Network

- Each node i holds **local model parameters** $\mathbf{w}^{(i)} \in \mathbb{R}^d$
- We stack them into one large vector:

$$\mathbf{w} = \left(\mathbf{w}^{(1)},\ \mathbf{w}^{(2)}\right) \in \mathbb{R}^{2d}$$

- Measure usefulness of $\mathbf{w}$ by local loss functions PLUS generalized total variation (GTV) penalty

One Possible GTVMin for Our Toy Network



Each node trains **linear model** via MSE:

$$L_i\left(\mathbf{w}^{(i)}\right) = \frac{1}{m_i}\left\|\mathbf{X}^{(i)}\mathbf{w}^{(i)} - \mathbf{y}^{(i)}\right\|_2^2$$

The **GTVMin objective** couples both nodes via a penalty on parameter disagreement:

$$\min_{\mathbf{w}^{(1)},\,\mathbf{w}^{(2)}} \underbrace{L_1\left(\mathbf{w}^{(1)}\right) + L_2\left(\mathbf{w}^{(2)}\right)}_{\text{local losses}} + \underbrace{\alpha\left\|\mathbf{w}^{(1)} - \mathbf{w}^{(2)}\right\|_2^2}_{\text{GTV penalty}}$$

- $\alpha = 0$: nodes train independently (no collaboration)
- $\alpha \to \infty$: forces $\mathbf{w}^{(1)} = \mathbf{w}^{(2)}$ (single global model)

Toy FL Network in Python

```
G = nx.Graph()
G.add_node(0, w=np.zeros(2), X=X1, y=y1)
G.add_node(1, w=np.zeros(2), X=X2, y=y2)
G.add_edge(0, 1)
```

Each node stores:

- `w` – model parameters $\mathbf{w}^{(i)}$
- `X` – feature matrix $\mathbf{X}^{(i)}$
- `y` – label vector $\mathbf{y}^{(i)}$

A FL-Design Principle/Workflow

1. **Formulate** the GTVMin optimization problem for your FL network
2. **Find** operator $\mathcal{F}$ such that the optimal $\widehat{\mathbf{w}}$ satisfies $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$
3. **Verify** $\mathcal{F}$ is a contractive operator (guarantees unique solution and convergence)
4. **Iterate** $\mathbf{w}^{(t+1)} = \mathcal{F}\big(\mathbf{w}^{(t)}\big)$

Recap: Contraction and Banach's Theorem

An operator $\mathcal{F}$ is a contractive operator if there exists $\kappa < 1$ such that

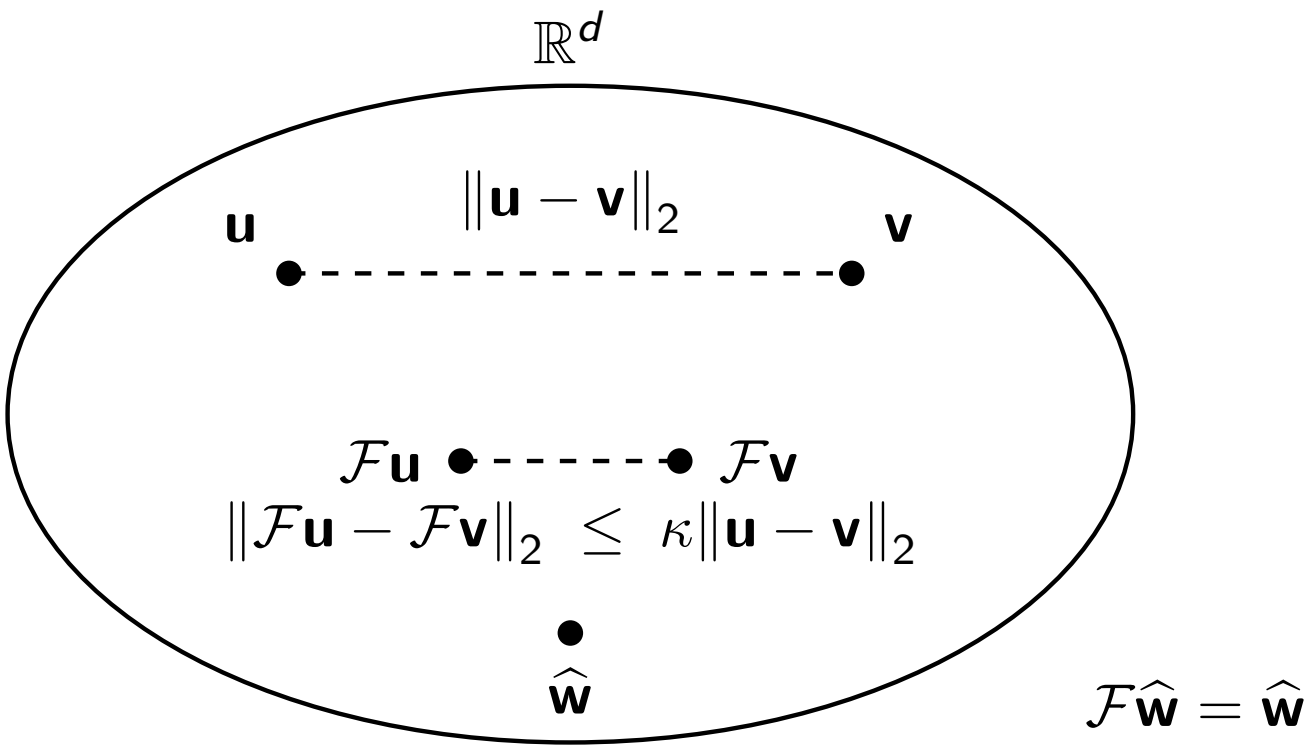
$$\|\mathcal{F}(\mathbf{u}) - \mathcal{F}(\mathbf{v})\|_2 \leq \kappa \|\mathbf{u} - \mathbf{v}\|_2 \quad \text{for all } \mathbf{u}, \mathbf{v}.$$

Banach's Fixed-Point Theorem:

- There exists a **unique** fixed point $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$
- Iterates converge with **geometric rate**: $\left\|\mathbf{w}^{(t)} - \widehat{\mathbf{w}}\right\|_2 \leq \kappa^t \left\|\mathbf{w}^{(0)} - \widehat{\mathbf{w}}\right\|_2$
- Smaller $\kappa \Rightarrow$ faster convergence

Key insight: if we can show $\mathcal{F}$ is a contractive operator, we get an iterative FL algorithm (which uses update $\mathbf{w}^{(t+1)} = \mathcal{F}\mathbf{w}^{(t)}$).

Contraction: Geometric Picture



A contractive operator **shrinks distances**: images $\mathcal{F}\mathbf{u}$, $\mathcal{F}\mathbf{v}$ are always closer than the inputs $\mathbf{u}$, $\mathbf{v}$. The unique point that does not move is the fixed point $\widehat{\mathbf{w}}$.

Recap: GD for Linear Regression

Single device with loss $L(\mathbf{w}) = \frac{1}{m}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$.

- **GD step:** $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{2\eta}{m}\mathbf{X}^T(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$
- **As affine map:** $\mathcal{F}(\mathbf{w}) = \mathbf{F}\,\mathbf{w} + \mathbf{b}, \quad \mathbf{F} = \mathbf{I} - \frac{2\eta}{m}\mathbf{X}^T\mathbf{X}$
- **Contraction iff** $\|\mathbf{F}\|_2 < 1 \;\Leftrightarrow\; \eta < \frac{m}{\lambda_{\max}(\mathbf{X}^T\mathbf{X})}$
- **Contraction rate:**
  $\kappa = \|\mathbf{F}\|_2 = \max\left(\left|1 - \frac{2\eta}{m}\lambda_{\min}\right|,\ \left|1 - \frac{2\eta}{m}\lambda_{\max}\right|\right)$

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

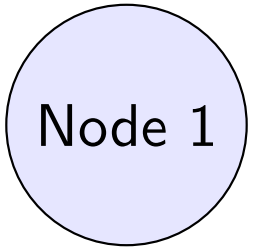
Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

The Simplest FL Network

Simplest possible FL network: **one node, no edges**.



- No neighbours $\Rightarrow$ no communication, no penalty term
- GTVMin reduces to plain **empirical risk minimization**:

$$\min_{\mathbf{w} \in \mathbb{R}^d} L(\mathbf{w})$$

- The workflow (formulate $\rightarrow$ fixed-point $\rightarrow$ contraction $\rightarrow$ iterate) still applies – let's walk through it with real data.

Demo 1: Setup

Data: The r-th data point is one day in Finland with

- feature $x^{(r)}$ = daily minimum temperature
- label $y^{(r)}$ = daily maximum temperature

Model and loss:

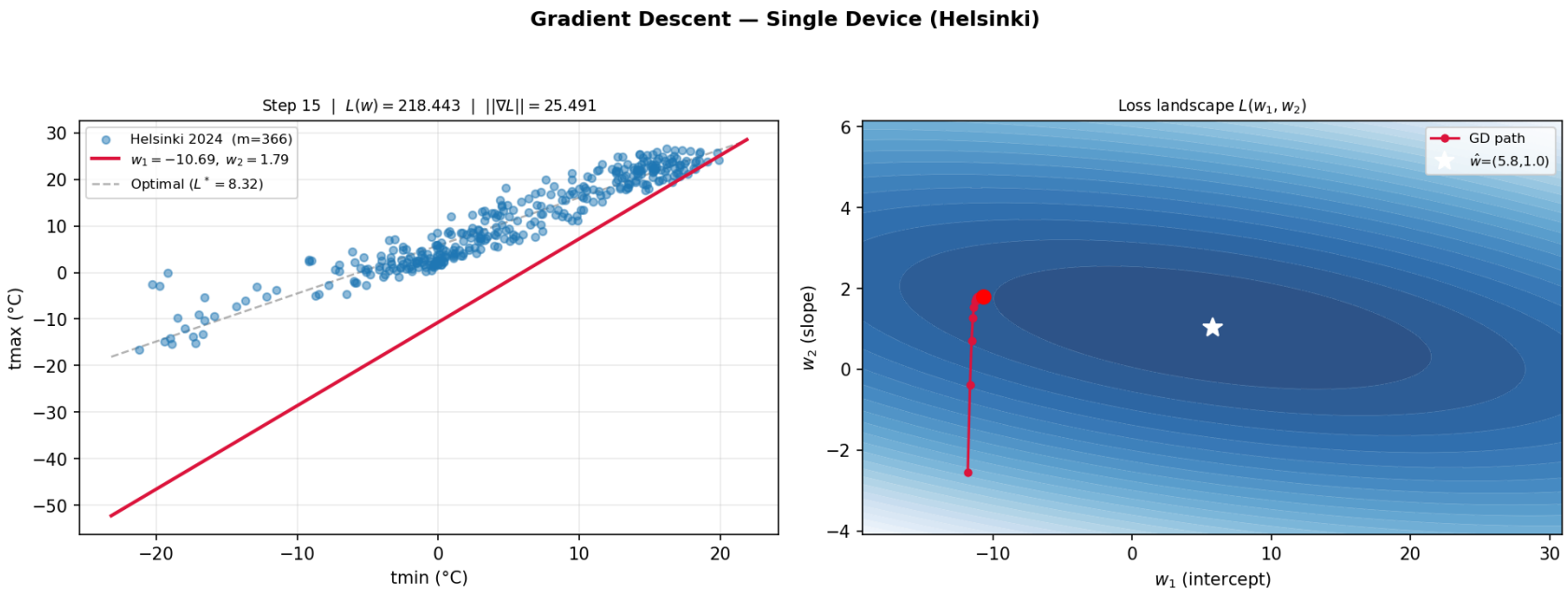
We train a linear model $\hat{y} = w_1 + w_2 \cdot x$ by minimizing the mean squared error (MSE):

$$L(\mathbf{w}) = \frac{1}{m}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

with feature matrix $\mathbf{X} = \left(\mathbf{1}, \mathbf{x}\right) \in \mathbb{R}^{m \times 2}$, $\mathbf{x} = \left(x^{(1)}, \ldots, x^{(m)}\right)^T$, and label vector $\mathbf{y} = \left(y^{(1)}, \ldots, y^{(m)}\right)^T$.

Run `python demos/Session3_Demo1.py` and press Enter to step through GD.

Demo 1: Demo Output



Left: data scatter + current hypothesis line. **Right:** loss contours $L(w_1, w_2)$ + GD trajectory.
Each Enter press performs one gradient descent (GD) step.

Demo 1: Console Output

```
360 valid days
lam_min(X'X) = 282.96   lam_max(X'X) = 39229.57
Condition number kappa = 138.64
eta = 0.002316  (ETA_FRAC=0.25 of optimal)
Contraction rate ~ 0.9856
Optimal weights: w1 = 5.779, w2 = 1.029
Optimal loss   : 8.3248

============================================================
 Gradient Descent Demo
  eta = 0.002316  |  starting at w = [-10.38, -3.68]
  Enter -- next step  |  r + Enter -- restart  |  Ctrl-C
  -- quit
============================================================

[step 1]  L=1087.44  ||grad||=203.84  ->  Enter
  w1=-9.95  w2=0.80  L=276.58  ||grad||=52.35
[step 2]  L=276.58   ||grad||=52.35   ->  Enter
  w1=-9.30  w2=0.95  L=205.01  ||grad||=31.32
[step 3]  L=205.01   ||grad||=31.32   ->  Enter
  w1=-8.71  w2=0.99  L=188.50  ||grad||=27.61
```

Demo 1: What the Console Tells Us

- $\lambda_{\min}(\mathbf{X}^T\mathbf{X}) \approx 283, \quad \lambda_{\max}(\mathbf{X}^T\mathbf{X}) \approx 39\,230 \;\Rightarrow\; \kappa \approx 139$ (ill-conditioned, raw features)
- Learning rate $\eta \approx 0.0023$, guarantees $\|\mathbf{F}\|_2 < 1$
- Contraction rate ≈ 0.99, GD converges slowly
- L and $\|\nabla L\|$ shrink every step:

Step	$L(\mathbf{w})$	$\|\nabla L\|$
0	1087	204
1	277	52

- Convergence is **guaranteed** but needs many steps – Demo 2 will show how normalization fixes this.

Demo 1: Connecting Theory to Code

Theory	**Code** (`Session3_Demo1.py`)
$\mathbf{X} \in \mathbb{R}^{m \times d}$	`X = np.column_stack([ones, tmin])`
$\nabla L = \frac{2}{m}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$	`(2/m) * X.T @ (X @ w - y)`
$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L$	`w - ETA * grad(w)`
$\lambda_{\max}(\mathbf{X}^T\mathbf{X})$	`np.linalg.eigvalsh(X.T @ X)`
$\eta < \frac{m}{\lambda_{\max}}$	`ETA_FRAC * m / (lam_max + lam_min)`

Every line of theory from Session 2 maps to one line of NumPy.

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

Demo 2: Why Normalize Features?

In Demo 1 we used raw tmin values. The condition number was $\kappa \approx 139$.

Recall: the contraction rate of GD depends on $\kappa = \lambda_{\text{max}}(\mathbf{X}^T\mathbf{X})/\lambda_{\text{min}}(\mathbf{X}^T\mathbf{X})$:

$$\kappa_{\text{rate}} = \frac{\kappa - 1}{\kappa + 1}$$

Large $\kappa \;\Rightarrow\; \kappa_{\text{rate}} \approx 1 \;\Rightarrow$ slow convergence.

Idea: standardize each feature to zero mean, unit variance:

$$\tilde{x}^{(r)} = \frac{x^{(r)} - \bar{x}}{\sigma_x}$$

This makes both columns of $\mathbf{X}$ have comparable scale $\Rightarrow$ $\mathbf{X}^T\mathbf{X} \approx m\,\mathbf{I} \;\Rightarrow\; \kappa \approx 1$.

Demo 2: Experiment

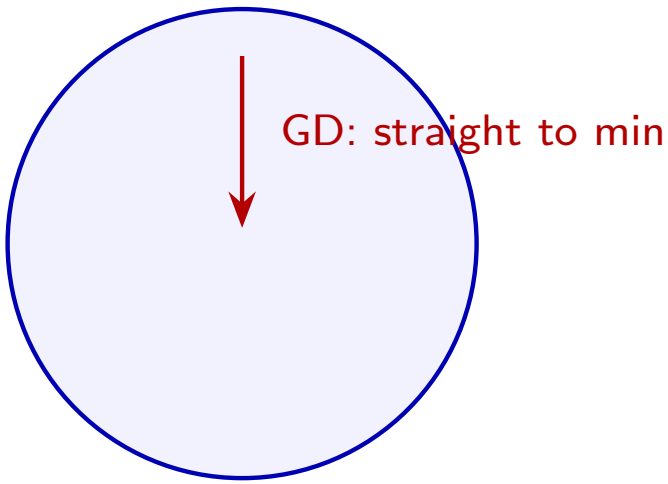
Experiment: modify `Session3_Demo1.py`:

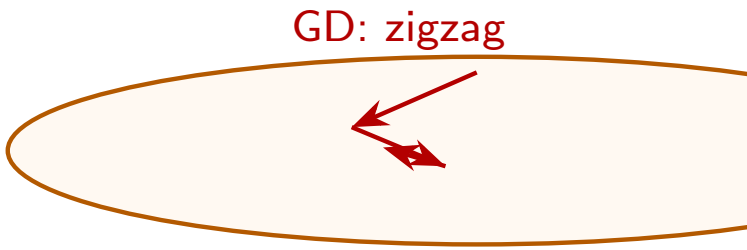
1. **Run as-is** (raw features).
   - $\kappa \approx 139$, contraction rate ≈ 0.99
   - GD needs many steps to converge

2. **Add standardization:** replace `tmin` in the design matrix with `(tmin - tmin.mean()) / tmin.std()`.
   - $\kappa \approx 1.3$, contraction rate ≈ 0.13
   - GD converges in a few steps
   - Loss contours become nearly circular

3. **Try a large learning rate:** set `ETA_FRAC = 1.5`. Does GD still converge?

Condition Number: Geometric Intuition





$\kappa \approx 1$: circular contours

$\kappa \gg 1$: elongated contou

	Raw features	**Standardized**
$\kappa(\mathbf{X}^T\mathbf{X})$	≈ 139	≈ 1.3
Contraction rate	≈ 0.99	≈ 0.13
Contour shape	elongated ellipse	near-circle

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

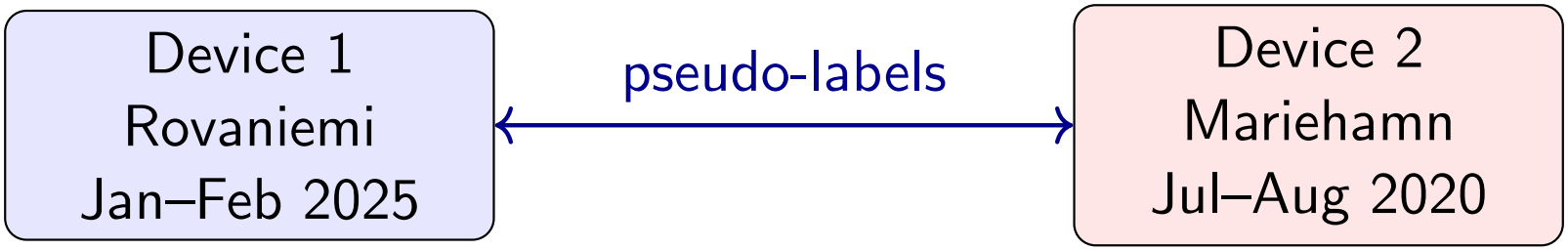
Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

Demo 3: Setup

Task: two weather stations collaborate via federated learning.



- Device 1: cold regime (-30 to 0 °C), Decision Tree
- Device 2: warm regime (10 to 30 °C), Linear Regression
- Neither device has seen the other's temperature range

Run `python demos/two_device_fl.py` and step through with Enter.

Demo 3: The Four Steps

1. **Local training.** Each device fits a model on its private data. $\Rightarrow$ good locally, blind outside its range.

2. **Synthetic feature generation.** Each device fits $\mathcal{N}(\mu, \sigma^2)$ to its local tmin and draws synthetic samples.

3. **Pseudo-label exchange.** Device labels synthetic features with its model, sends $(X_{\text{synth}}, \hat{y})$ to its neighbour. $\Rightarrow$ no raw data leaves the device!

4. **Augmented re-training.** Each device pools local data + received pseudo-labels and re-fits. $\Rightarrow$ model improves in previously unseen regime.

Repeat steps 2–4 for `N_ITER` iterations.

Demo 3: What to Observe

- After Step 1: each model is accurate *only* in its local range
- After Step 3: the $\triangle$ markers show pseudo-labeled data received from the peer – they extend coverage into new regimes
- After Step 4: the updated model curve stretches to cover *both* temperature regimes
- Over iterations: RMSE drops as devices refine each other's models

Questions:

- What happens if you increase `N_SYNTH` from 40 to 200?
- What if both devices use the same model type?
- How does this relate to the **read–update–share** blueprint?

Demo 3: FL Network in Code

```
import networkx as nx

# FL network: two nodes connected by one edge
G = nx.Graph()
G.add_node(0, device={"place": "Rovaniemi", ...})
G.add_node(1, device={"place": "Mariehamn", ...})
G.add_edge(0, 1)

# Core loop (one iteration)
for n in G.nodes:
    dev = G.nodes[n]["device"]

    # 1. READ: get pseudo-labels from neighbours
    for nb in G.neighbors(n):
        nb_dev = G.nodes[nb]["device"]
        recv = (nb_dev["X_synth"], nb_dev["y_pseudo"])

    # 2. UPDATE: re-train on local + received data
    dev["model"].fit(X_augmented, y_augmented)

    # 3. SHARE: generate new pseudo-labels for next round
    dev["y_pseudo"] = dev["model"].predict(dev["X_synth"])
```

Table of Contents

Session 2 Feedback

Quick Recap: The FL Algorithm Workflow

Demo 1: GD on a Single Device

Demo 2: Feature Normalization and Condition Number

Demo 3: Two-Device FL

Summary and Next Steps

Summary

- **Demo 1:** every GD step is an application of the affine operator $\mathcal{F}(\mathbf{w}) = \mathbf{F}\mathbf{w} + \mathbf{b}$ – one line of NumPy per theory equation
- **Demo 2:** normalizing features reduces κ and the contraction rate – GD converges much faster
- **Demo 3:** FL = devices running **read–update–share** on a networkx graph – no raw data leaves a device

Next session (10 April): Federated Clustering.
Self-study: read `LegalFrameworksFL_IMC.pdf` (GDPR & EU AI Act).
Post questions on the Teams channel anytime.